

Filed under *on November 21st, 2023*

Last updated on November 13th, 2025



CSS Grid is one of the most amazing parts of the CSS language. It gives us a ton of new tools we can use to create sophisticated and fluid layouts.

It's also *surprisingly complex*. It took me quite a while to truly become comfortable with CSS Grid!

In this tutorial, I'm going to share the biggest 💡 lightbulb moments I've had in my own journey with CSS Grid. You'll learn the fundamentals of this layout mode, and see how to do some pretty cool stuff with it. ✨

Browser support?

CSS Grid is the most modern tool for building layouts in CSS, but it isn't exactly “bleeding edge”. It's been supported by all major browsers since 2017!

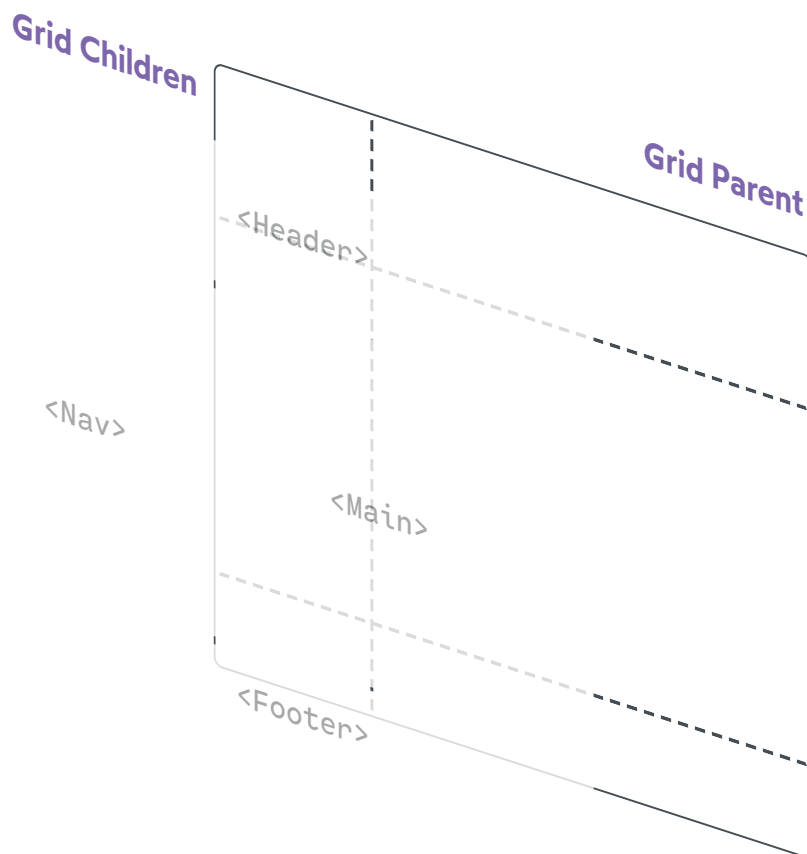
According to caniuse, CSS Grid is [_____](#). This is *excellent* browser support; to put that number in context, Flexbox is only about 0.3% more supported!

Mental model

CSS is comprised of several different _____, each designed for different types of user interfaces. The default layout algorithm, Flow layout, is designed for digital documents. Table layout is designed for tabular data. Flexbox is designed for distributing items along a single axis.

CSS Grid is the latest and greatest layout algorithm. It's *incredibly* powerful: we can use it to build complex layouts that fluidly adapt based on a number of constraints.

The most unusual part of CSS Grid, in my opinion, is that the grid *structure*, the rows and columns, are defined **purely in CSS**:



Show Perspective

With CSS Grid, a single DOM node is sub-divided into rows and columns. In this tutorial, we're highlighting the rows/columns with dashed lines, but in reality, they're invisible.

This is super weird! In every other layout mode, the only way to create compartments like this is by adding more DOM nodes. In Table layout, for example, each row is created with a `<tr>`, and each cell within that row using `<td>` or `<th>`:

```
<table>
  <tbody>
```

```
<!-- First row -->
<tr>
  <!-- Cells in the first row -->
  <td></td>
  <td></td>
  <td></td>
</tr>

<!-- Second row -->
<tr>
  <!-- Cells in the second row -->
  <td></td>
  <td></td>
  <td></td>
</tr>
</tbody>
</table>
```

Unlike Table layout, CSS Grid lets us manage the layout entirely from within CSS. We can slice up the container however we wish, creating compartments that our grid children can use as anchors.

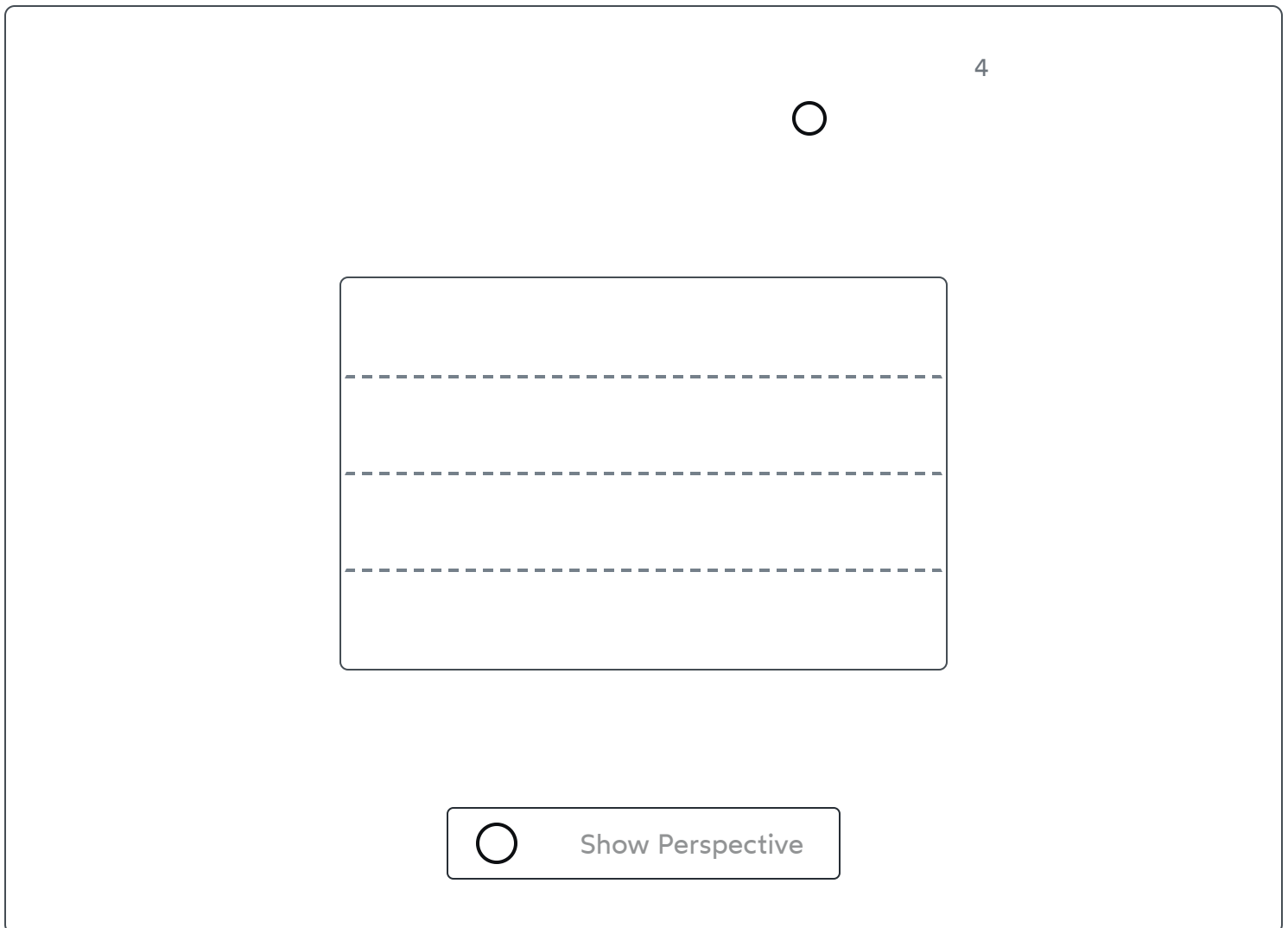
Grid flow

We opt in to the Grid layout mode with the `display` property:

```
.wrapper {
  display: grid;
}
```

By default, CSS Grid uses a single column, and will create rows as needed, based on the number of children. This is known as an *implicit grid*, since we aren't explicitly defining any structure.

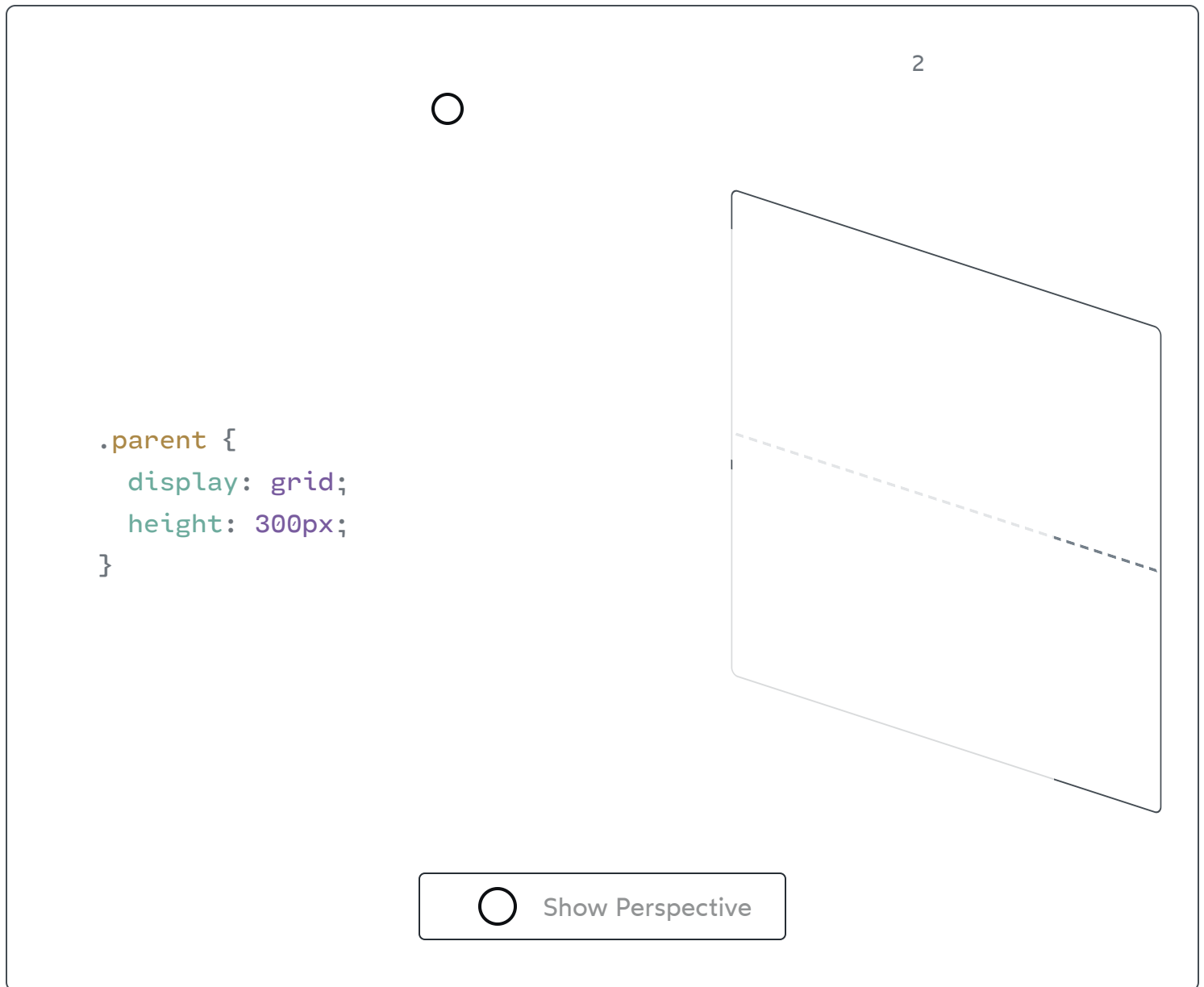
Here's how this works:



Implicit grids are dynamic; rows will be added and removed based on the number of children. Each child gets its own row.

By default, the height of the grid parent is determined by its children. It grows and shrinks dynamically. Interestingly, this isn't even a “CSS Grid” thing; the grid *parent* is still using Flow layout, and block elements in Flow layout grow vertically to contain their content. Only the *children* are arranged using Grid layout.

But what if we give the grid a fixed height? In that case, the total surface area is divided into equally-sized rows:



Grid Construction

By default, CSS Grid will create a single-column layout. We can specify columns using the `grid-template-columns` property:

HTML CSS

```
<style>
  .parent {
    display: grid;
    grid-template-columns: 25% 75%;
  }
</style>

<div class="parent">
  <div class="child">1</div>
  <div class="child">2</div>
</div>
```

RESULT



Dashed lines added for clarity

In the playground above, and throughout this tutorial, I'm using dashed lines to show the divisions between columns and rows. In CSS grid, these lines are invisible, and can't be made visible.

I'm using some ✨ *blog magic* ✨ here, faking it with pseudo-elements. You can pop over to the **CSS** tab if you'd like to see how.

By passing two values to `grid-template-columns` — `25%` and `75%` — I'm telling the CSS Grid algorithm to slice the element up into two columns.

Columns can be defined using any valid [length](#), including pixels, rems, viewport units, and so on. Additionally, we also gain access to a new unit, the `fr` unit:

HTML CSS

```
<style>
  .parent {
    display: grid;
    grid-template-columns: 1fr 3fr;
  }
</style>

<div class="parent">
  <div class="child">1</div>
  <div class="child">2</div>
</div>
```

RESULT



`fr` stands for “fraction”. In this example, we're saying that the first column should consume 1 unit of space, while the second column consumes 3 units of space. That means there are 4 total units of space, and this becomes the denominator. The first column eats up $\frac{1}{4}$ of the available space, while the second column consumes $\frac{3}{4}$.

The `fr` unit brings Flexbox-style flexibility to CSS Grid. Percentages and `<length>` values create hard constraints, while `fr` columns are free to grow and shrink as required, to contain their contents.

Try shrinking this container to see the difference:

40%

☐ fr ☐ % ☐

```
.parent {  
  display: grid;  
  grid-template-columns: 1fr 3fr;  
}
```

In this scenario, our first column has a cuddly ghost that has been given an explicit width of 55px. But what if the column is too small to contain it?

- Percentage-based columns are rigid, and so our ghost image will *overflow*, spilling out of the column.
- `fr`-based columns are flexible, and so the column won't shrink below its minimum content size, even if that means breaking the proportions.

To be more precise: the `fr` unit distributes *extra* space. First, column widths will be calculated based on their contents. If there's any leftover space, it'll be

distributed based on the `fr` values. This is very similar to `flex-grow`, as discussed in my [article](#).

In general, this flexibility is a good thing. Percentages are too strict.

We can see a perfect example of this with `gap`. `gap` is a magical CSS property that adds a fixed amount of space between all of the columns and rows within our grid.

Check out what happens when we toggle between percentages and fractions:

Unit:

Fractions

Percentages



```
.parent {  
  display: grid;  
  grid-template-columns: 1fr 3fr;  
  gap: 16px;  
}
```

Notice how the contents spill outside the grid parent when using percentage-based columns? This happens because percentages are calculated using the *total*

grid area. The two columns consume 100% of the parent's content area, and they aren't allowed to shrink. When we add 16px of `gap`, the columns have no choice but to spill beyond the container.

The `fr` unit, by contrast, is calculated based on the *extra* space. In this case, the extra space has been reduced by 16px, for the `gap`. The CSS Grid algorithm distributes the remaining space between the two grid columns.

vs.

When CSS Grid was first introduced, the `column-gap` property was used to add space between columns and rows. Pretty quickly, however, the community realized that this feature would be *awesome* to have in Flexbox as well. And so, the property was given a more-generic name, `column-gap`.

These days, `column-gap` has been marked as deprecated, and browsers have aliased it to `column-gap`. Both properties do the exact same thing. And they both have nearly-identical browser support, [see here](#).

And so, I recommend using `column-gap` rather than `column-gap`, whether you're working with Flexbox or CSS Grid. But there's also no urgency when it comes to converting existing declarations.

Implicit and explicit rows

What happens if we add more than two children to a two-column grid?

Well, let's give it a shot:

```
HTML CSS
<style>
  .parent {
    display: grid;
```

```
    grid-template-columns: 1fr 3fr;
  }
</style>

<div class="parent">
  <div class="child">1</div>
  <div class="child">2</div>
```



RESULT



Interesting! Our grid gains a second row. The grid algorithm wants to ensure that every child has its own grid cell. It'll spawn new rows as-needed to fulfill this goal. This is handy in situations where we have a variable number of items (eg. a photo grid), and we want the grid to expand automatically.

In other situations, though, we want to define the rows explicitly, to create a specific layout. We can do that with the `grid-template-rows` property:

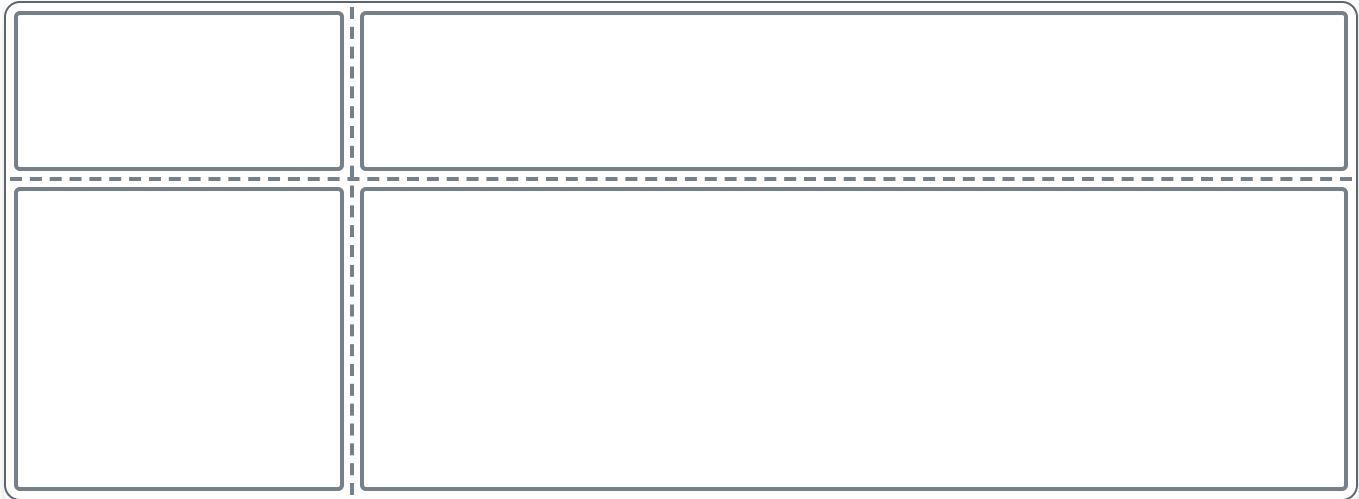
HTML CSS

```
<style>
  .parent {
    display: grid;
    grid-template-columns: 1fr 3fr;
    grid-template-rows: 5rem 1fr;
  }
</style>

<div class="parent">
```

```
<div class="child"></div>
<div class="child"></div>
<div class="child"></div>
<div class="child"></div>
</div>
```

RESULT



By defining both `grid-template-rows` and `grid-template-columns`, we've created an explicit grid. This is perfect for building page layouts, like the “Holy Grail”[?] layout at the top of this tutorial.

The repeat helper

Let's suppose we're building a calendar:



CSS Grid is a wonderful tool for this sort of thing. We can structure it as a 7-column grid, with each column consuming 1 unit of space:

```
.calendar {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr 1fr 1fr 1fr 1fr;  
}
```

This *works*, but it's a bit annoying to have to count each of those `1fr` 's. Imagine if we had 50 columns!

Fortunately, there's a nicer way to solve for this:

```
.calendar {  
  display: grid;  
  grid-template-columns: repeat(7, 1fr);  
}
```

The `repeat` function will do the copy/pasting for us. We're saying we want 7 columns that are each `1fr` wide.

Here's the playground showing the full code, if you're curious:

HTML CSS

```
<style>  
  .calendar {
```

```
display: grid;
grid-template-columns:
  repeat(7, 1fr);
gap: 4px;
}
</style>
```

```
<!--
```

Normally, I'd use something like React to generate these cells dynamically, but I'm using plain HTML here to keep things simple.

```
-->
```

```
<ol class="calendar">
  <li class="day">1</li>
  <li class="day">2</li>
  <li class="day">3</li>
  <li class="day">4</li>
  <li class="day">5</li>
  <li class="day">6</li>
  <li class="day">7</li>
  <li class="day">8</li>
  <li class="day">9</li>
  <li class="day">10</li>
  <li class="day">11</li>
  <li class="day">12</li>
  <li class="day">13</li>
  <li class="day">14</li>
  <li class="day">15</li>
  <li class="day">16</li>
  <li class="day">17</li>
```

1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	31				

Calendars and accessibility

This calendar is a quick-and-dirty example of how we can use CSS Grid to create a specific layout. It isn't intended to be production-ready.

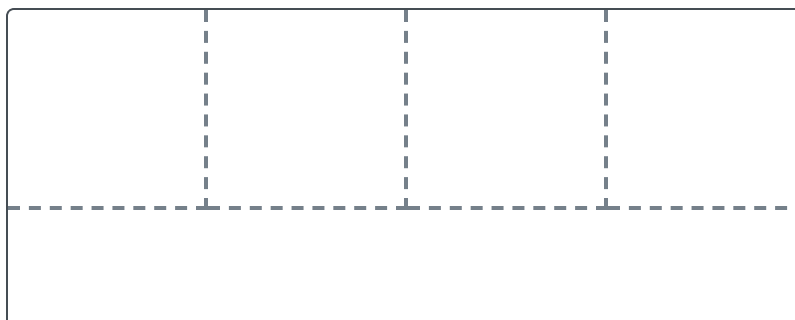
If you're looking to build a date-picker, you should absolutely use an accessibility-focused library like [date-fns](#).

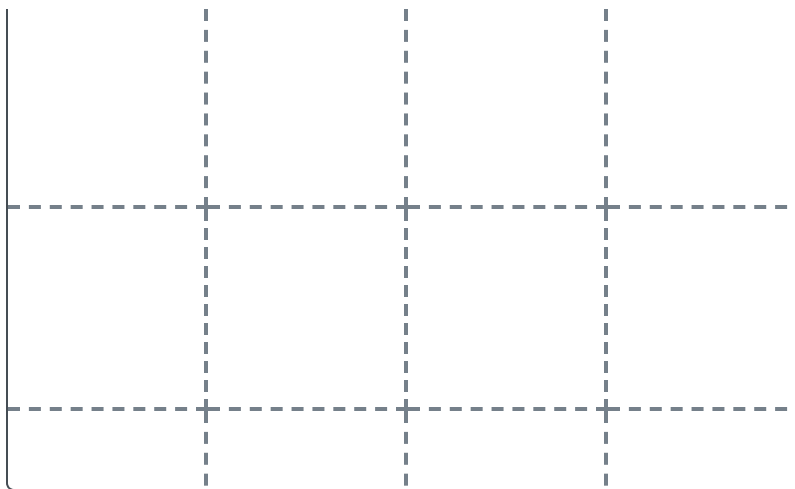
Assigning children

By default, the CSS Grid algorithm will assign each child to the first unoccupied grid cell, much like how a tradesperson might lay tiles in a bathroom floor.

Here's the cool thing though: we can assign our items to whichever cells we want! Children can even span across multiple rows/columns.

Here's an interactive demo that shows how this works. *Click/press and drag* to place a child in the grid :





```
.parent {  
  display: grid;  
  grid-template-columns:  
    repeat(4, 1fr);  
  grid-template-rows:  
    repeat(4, 1fr);  
}  
  
.child {  
  
}
```

The `grid-row` and `grid-column` properties allow us to specify which track(s) our grid child should occupy.

If we want the child to occupy a single row or column, we can specify it by its number. `grid-column: 3` will set the child to sit in the third column.

Grid children can also stretch across multiple rows/columns. The syntax for this uses a slash to delineate start and end:

```
.child {  
  grid-column: 1 / 4;  
}
```

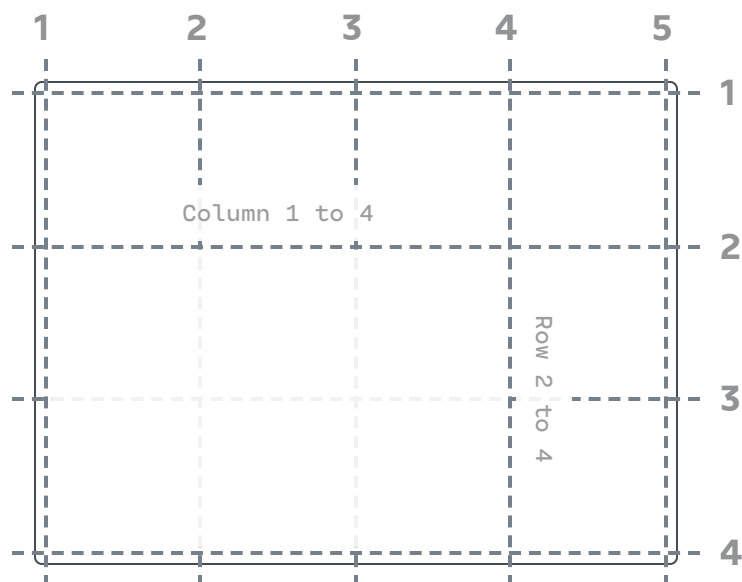
At first glance, this looks like a fraction, $\frac{1}{4}$. In CSS, though, the slash character is not used for division, it's used to separate groups of values. In this case, it allows us to set the start and end columns in a single declaration.

It's essentially a shorthand for this:

```
.child {  
  grid-column-start: 1;  
  grid-column-end: 4;  
}
```

There's a sneaky gotcha here: The numbers we're providing are based on the column *lines*, not the column indexes.

It'll be easiest to understand this gotcha with a diagram:



```
.child {  
  grid-row: 2 / 4;  
  grid-column: 1 / 4;  
}
```

Confusingly, a 4-column grid actually has 5 column lines. When we assign a child to our grid, we anchor them using these lines. If we want our child to span the first 3 columns, it needs to start on the 1st line and end on the 4th line.

Negative line numbers

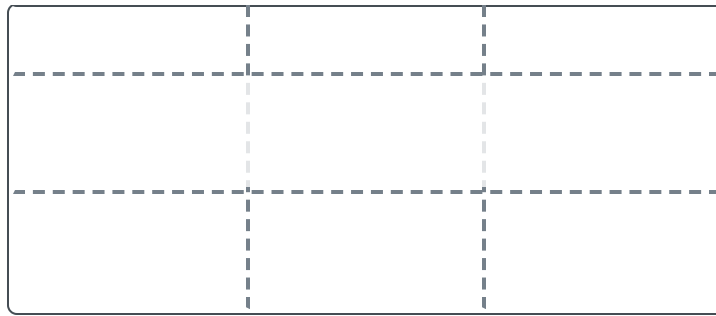
In a left-to-right language like English, we count the columns from left to right. With negative line numbers, however, we can also count in the *opposite* direction, from right to left.

```
.child {  
  /* Sit in the 2nd column from the right: */  
  grid-column: -2;  
}
```

The really cool thing is that we can mix positive and negative numbers. Check this out:

3





```
.parent {  
  display: grid;  
  grid-template-columns:  
    repeat(3, 1fr);  
}  
  
.child {  
  grid-column: 1 / -1;  
  grid-row: 2;  
}
```

Notice that the child spans the full width of the grid, even though we aren't changing the `grid-column` assignment at all!

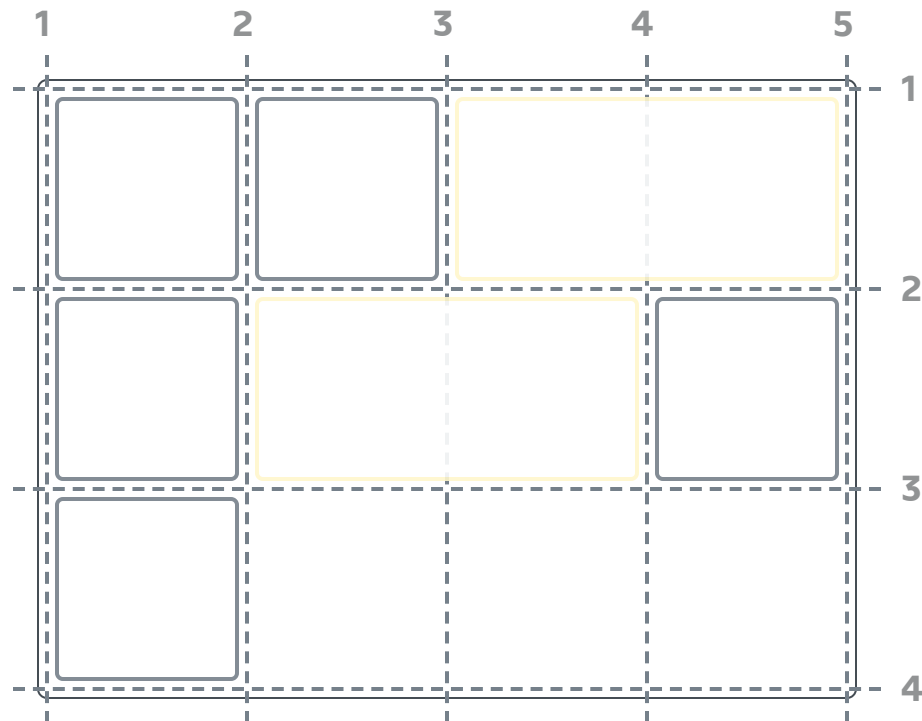
We're saying here that our child should span from the first column line to the last column line. No matter how many columns there are, this handy declaration will work as intended.

You can see a practical use case for this trick in my blog post, [_____](#).

Spanning multiple rows/columns

In the examples above, we used `grid-column` and `grid-row` to select a specific location for our grid child. This is useful when we know exactly where our child should sit, but in other cases, we'll want things to be a bit more dynamic.

For example, maybe we want our grid children to fill the grid naturally, but we want *certain* grid children to span multiple columns:



In this example, most of the grid children (represented with gray boxes) sit in a single grid cell, but we want *featured* children (represented with yellow boxes) to be double-wide, spanning across *two* columns.

We *could* solve this by giving each child a specific column/row assignment, but this sounds very tedious, especially if the grid is generated from dynamic data! Instead, we can use the special `span` keyword for featured children:

```
.grid {  
  display: grid;  
  grid-template-columns: repeat(4, 1fr);  
}
```

```
grid-template-rows: repeat(3, 1fr);  
}  
.featured.child {  
  grid-column: span 2;  
}
```

Here's a live-editable playground showcasing this functionality:

HTML CSS

```
<style>  
  .grid {  
    display: grid;  
    grid-template-columns: repeat(4, 1fr);  
    grid-template-rows: repeat(3, 1fr);  
  }  
  .featured.child {  
    grid-column: span 2;  
  }  
</style>  
  
<div class="grid">  
  <div class="child"></div>  
  <div class="child"></div>  
  <div class="featured child"></div>  
  <div class="child"></div>  
  <div class="featured child"></div>  
  <div class="child"></div>  
  <div class="child"></div>  
</div>
```

RESULT

Awkwardly-placed featured children?

In the example above, the two featured children are conveniently *not* assigned to the fourth and final column. What happens if we put a two-column-wide grid child in the last column??

Well, let's try it:

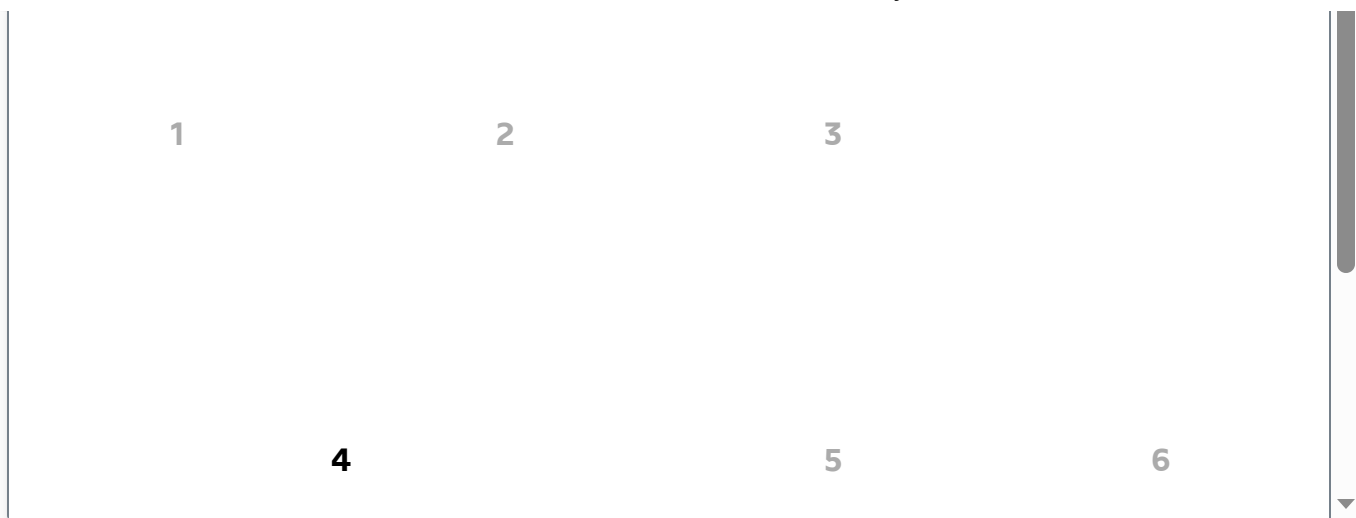


HTML CSS

```
<style>
  .featured.child {
    grid-column: span 2;
  }
</style>

<div class="grid">
  <div class="child">1</div>
  <div class="child">2</div>
  <div class="child">3</div>
  <!--
    A featured child in the 4th col:
  -->
  <div class="featured child">4</div>
  <div class="child">5</div>
  <div class="child">6</div>
</div>
```

RESULT



The featured child can't fit in that final column. As a result, it's pushed to the next row, and a blank spot is left in the first row's final cell.

CSS Grid has a neat trick for this sort of situation. If the *order* of the children isn't important, we can instruct the browser to cram the children in wherever they'll fit. Check this out:

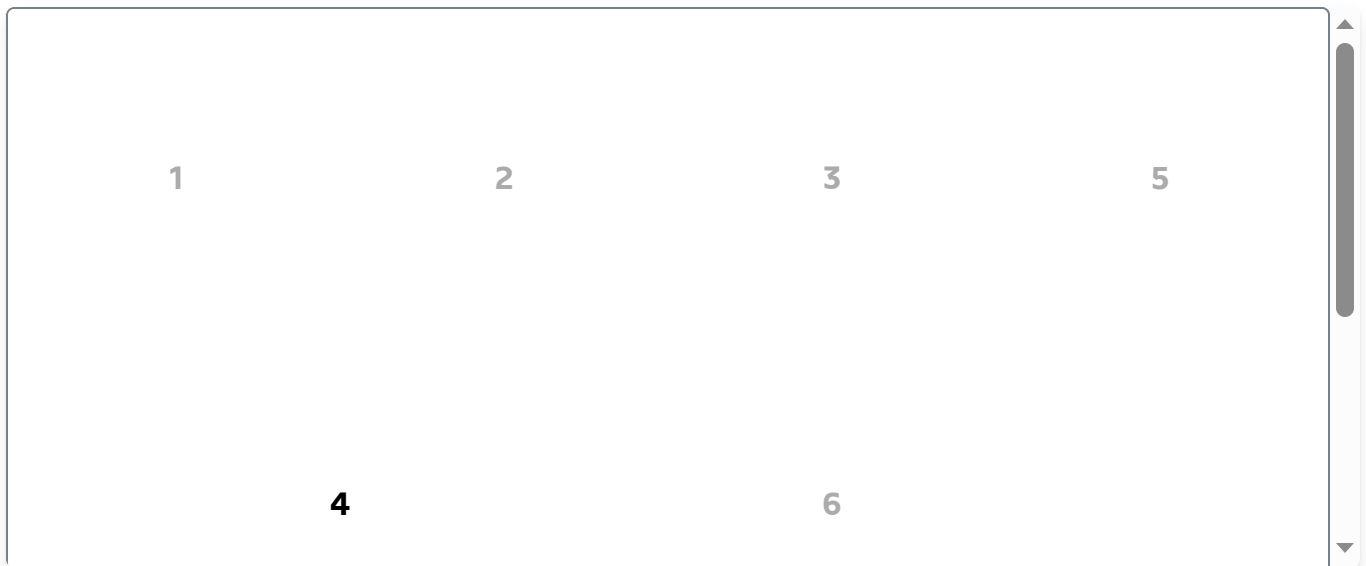


HTML CSS

```
<style>
  .grid {
    grid-auto-flow: row dense;
  }
  .featured.child {
    grid-column: span 2;
  }
</style>

<div class="grid">
  <div class="child">1</div>
  <div class="child">2</div>
  <div class="child">3</div>
  <div class="featured child">4</div>
  <div class="child">5</div>
  <div class="child">6</div>
</div>
```


RESULT

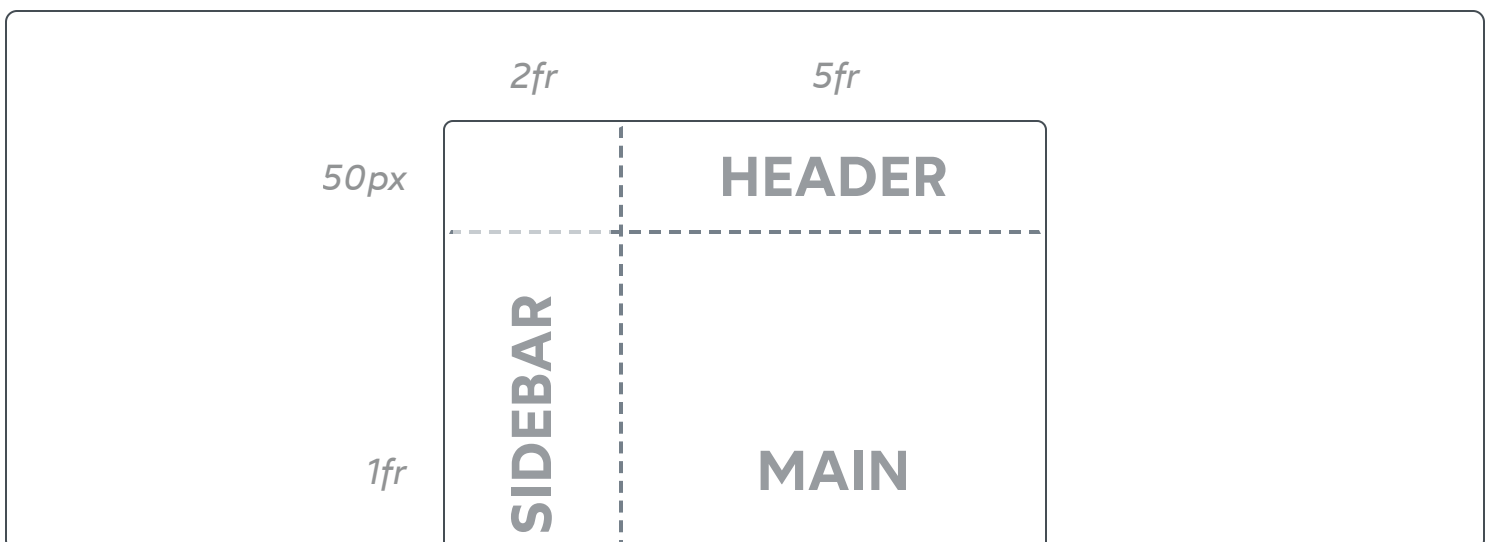


The `grid-auto-fit` property is a bit beyond the scope of this tutorial, but essentially it's designed to help with this sort of situation, to avoid holes left in our grid when dynamically assigning children of different sizes. In this case, it discovered that the fifth grid child *would* fit in that fourth grid cell, and so it gets plucked up and moved ahead of the featured child.

Grid areas

Alright, time to talk about one of the coolest parts of CSS Grid. 😊

Let's suppose we're building this layout:



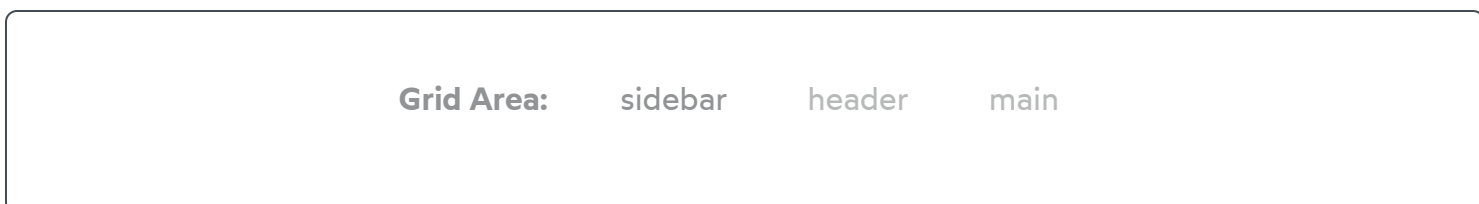


Using what we've learned so far, we could structure it like this:

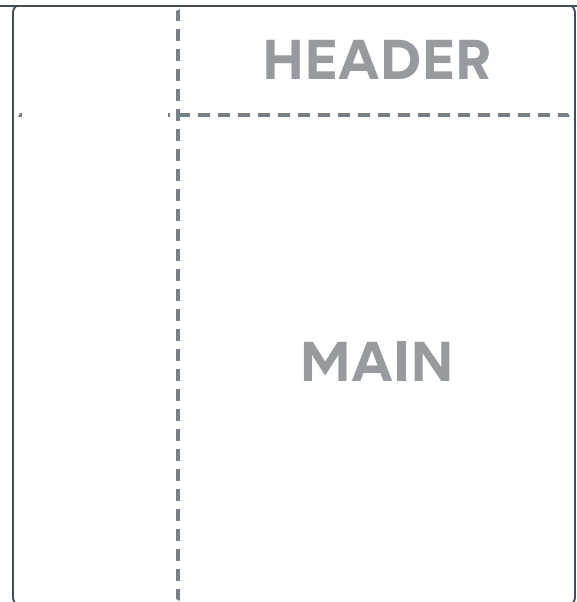
```
.grid {  
  display: grid;  
  grid-template-columns: 2fr 5fr;  
  grid-template-rows: 50px 1fr;  
}  
  
.sidebar {  
  grid-column: 1;  
  grid-row: 1 / 3;  
}  
  
.header {  
  grid-column: 2;  
  grid-row: 1;  
}  
  
.main {  
  grid-column: 2;  
  grid-row: 2;  
}
```

This works, but there's a more ergonomic way to do this: *grid areas*.

Here's what it looks like:



```
.parent {  
  display: grid;  
  grid-template-columns: 2fr 5fr;  
  grid-template-rows: 50px 1fr;  
  grid-template-areas:  
    'sidebar header'  
    'sidebar main';  
}  
  
.child {  
  grid-area: sidebar;  
}
```



Like before, we're defining the grid structure with `grid-template-columns` and `grid-template-rows`. But then, we have this curious declaration:

```
.parent {  
  grid-template-areas:  
    'sidebar header'  
    'sidebar main';  
}
```

Here's how this works: We're drawing out the grid we want to create, almost as if we were making ASCII art[?]. Each line represents a row, and each word is a name we're giving to a particular slice of the grid. See how it sorta looks like the grid, visually?

Then, instead of assigning a child with `grid-column` and `grid-row`, we assign it with `grid-area`!

When we want a particular area to span multiple rows or columns, we can repeat the name of that area in our template. In this example, the “sidebar” area spans both rows, and so we write `sidebar` for both cells in the first column.

Should we use areas, or rows/columns? When building explicit layouts like this, I really like using areas. It allows me to give semantic meaning to my grid assignments, instead of using inscrutable row/column numbers. That said, areas work best when the grid has a fixed number of rows and columns. `grid-column` and `grid-row` can be useful for implicit grids.

Being mindful of keyboard users

There's a big gotcha when it comes to grid assignments: **tab order will still be based on *DOM position*, not grid position.**

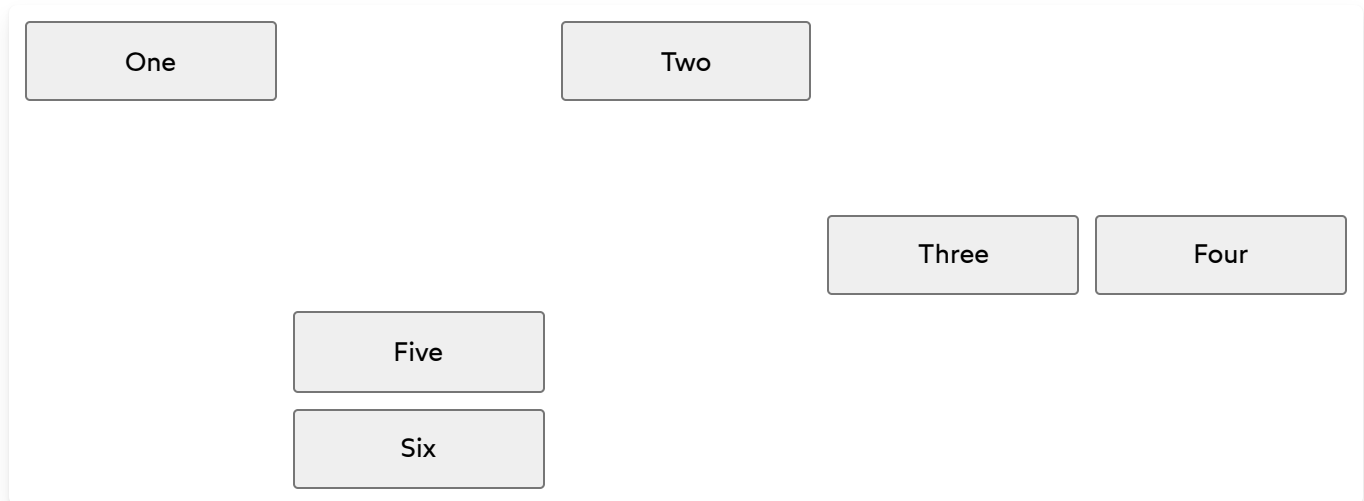
It'll be easier to explain with an example. In this playground, I've set up a group of buttons, and arranged them with CSS Grid:

HTML CSS

```
<div class="wrapper">
  <button class="btn one">
    One
  </button>
  <button class="btn four">
    Four
  </button>
  <button class="btn six">
    Six
  </button>
  <button class="btn two">
    Two
  </button>
  <button class="btn five">
    Five
  </button>
  <button class="btn three">
```

```
Three  
</button>  
</div>
```

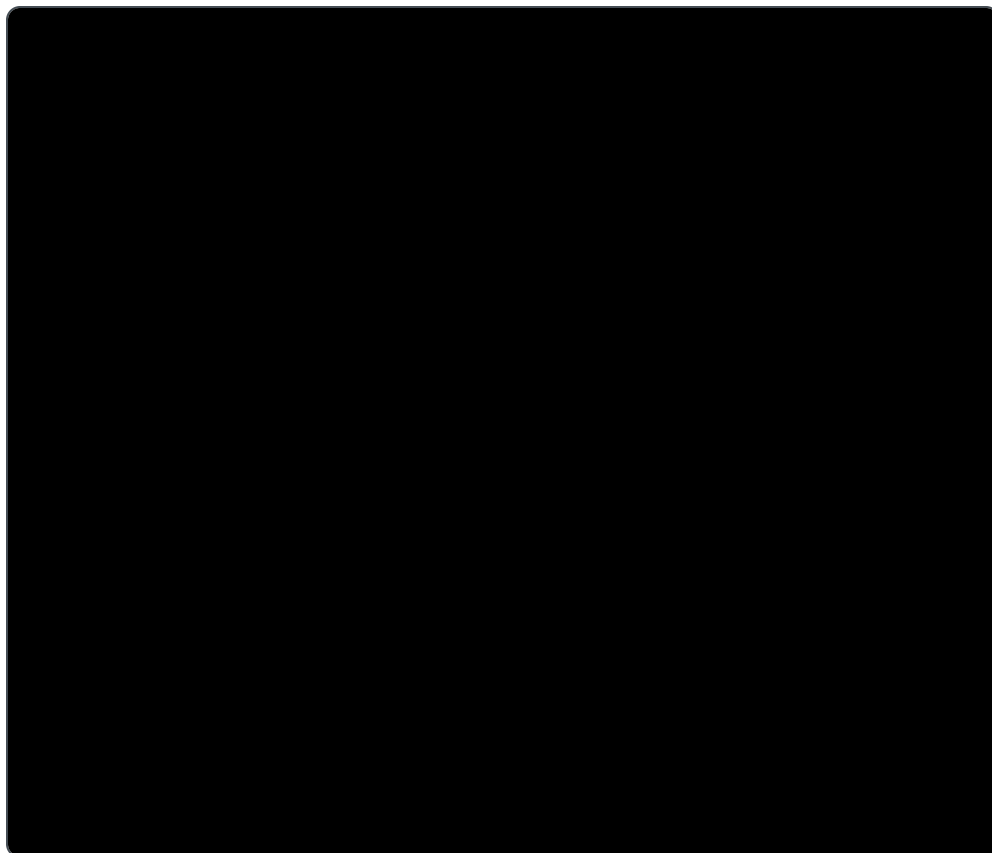
RESULT



In the “RESULT” pane, the buttons appear to be in order. By reading from left to right, and from top to bottom, we go from one to six.

If you're using a device with a keyboard, try to tab through these buttons. You can do this by clicking the first button in the top left (“One”), and then pressing Tab to move through the buttons one at a time.

You should see something like this:



The focus outline jumps around the page without rhyme or reason, from the user's perspective. This happens because the buttons are being focused based on the order they appear in the DOM.

To fix this, we should re-order the grid children in the DOM so that they match the visual order, so that I can tab through from left to right, and from top to bottom.

The `order` CSS property

The CSS Working Group is aware that this is kind of a flaw with CSS Grid. Having to match the DOM order with the visual order sorta defeats the purpose of being able to assign children to specific grid cells!

A fix is in the works. In the future, the `order` CSS property should allow us to instruct the browser to ignore the DOM order, and focus elements in the order they appear onscreen.

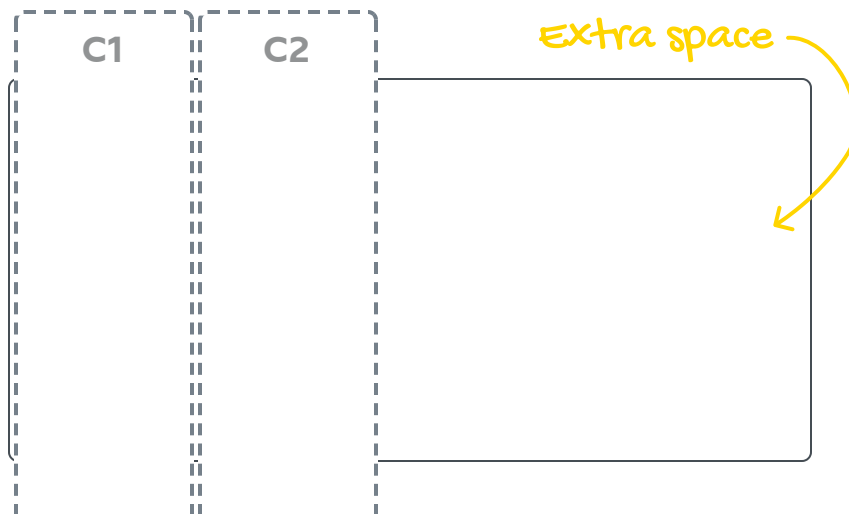
Unfortunately, this property is not yet implemented in any browsers, as of September 2024. You can learn more about this property in Rachel Andrew's post, [_____](#)



Alignment

In all the examples we've seen so far, our columns and rows stretch to fill the entire grid container. This doesn't need to be the case, however!

For example, let's suppose we define two columns that are each 90px wide. As long as the grid parent is larger than 180px, there will be some dead space at the end:



We can control the distribution of the columns using the `justify-content` property:

justify-content:

start

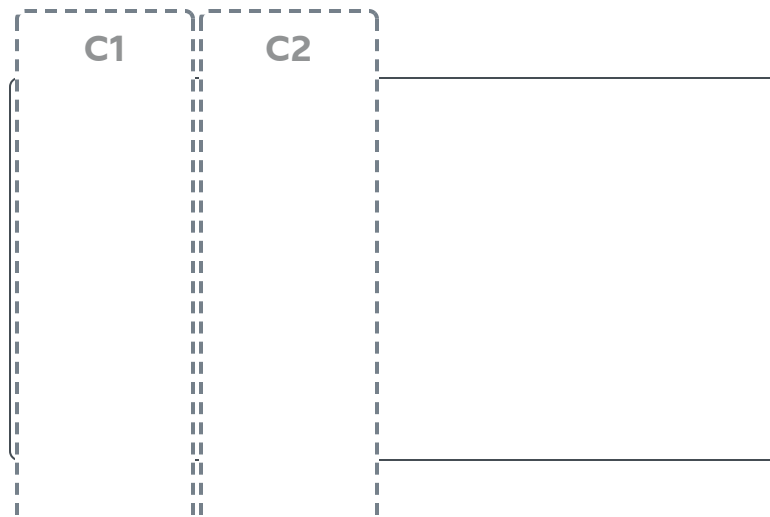
center

end

space-between

space-around

space-evenly



```
.parent {  
  display: grid;  
  grid-template-columns: 90px 90px;  
  justify-content: start;  
}
```



Show Perspective

If you're familiar with the Flexbox layout algorithm, this probably feels pretty familiar. CSS Grid builds on the alignment properties first introduced with Flexbox, taking them even further.

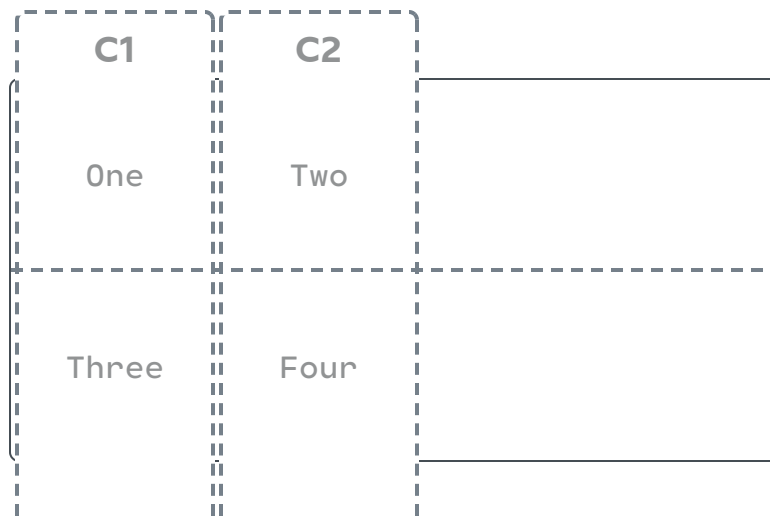
The big difference is that we're aligning the *columns*, not the items themselves. Essentially, `justify-content` lets us arrange the compartments of our grid, distributing them across the grid however we wish.

If we want to align the items themselves *within* their columns, we can use the `justify-items` property:

start (default)



stretch (default)



```
.parent {  
  display: grid;  
  grid-template-columns: 90px 90px;  
  justify-content: start;  
  justify-items: stretch;  
}
```



Show Perspective

When we plop a DOM node into a grid parent, the default behaviour is for it to stretch across that entire column, just like how a `<div>` in Flow layout will stretch horizontally to fill its container. With `justify-items`, however, we can tweak that behaviour.

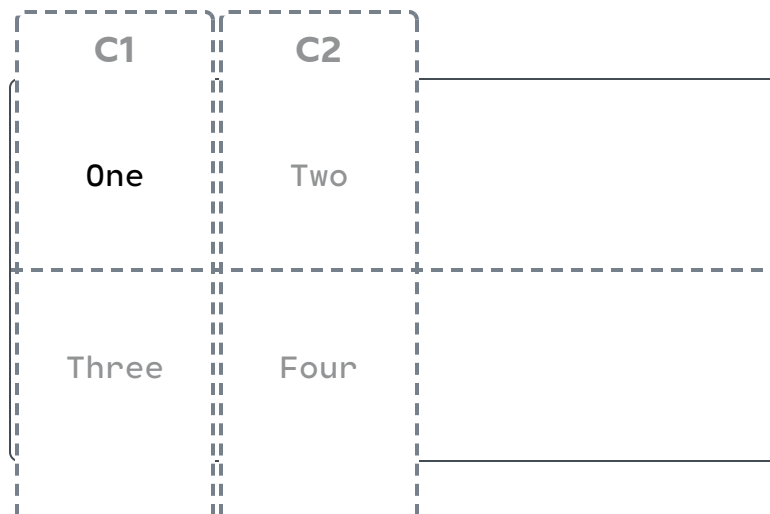
This is useful because it allows us to break free from the rigid symmetry of columns. When we set `justify-items` to something other than `stretch`, the children will shrink down to their default width, as determined by their contents. As a result, items in the same column can be different widths.

We can even control the alignment of a *specific* grid child using the `justify-self` property:

start (default)



stretch (default)



```
.parent {  
  display: grid;  
  grid-template-columns: 90px 90px;  
  justify-content: start;  
}  
  
.one {  
  justify-self: stretch;  
}
```

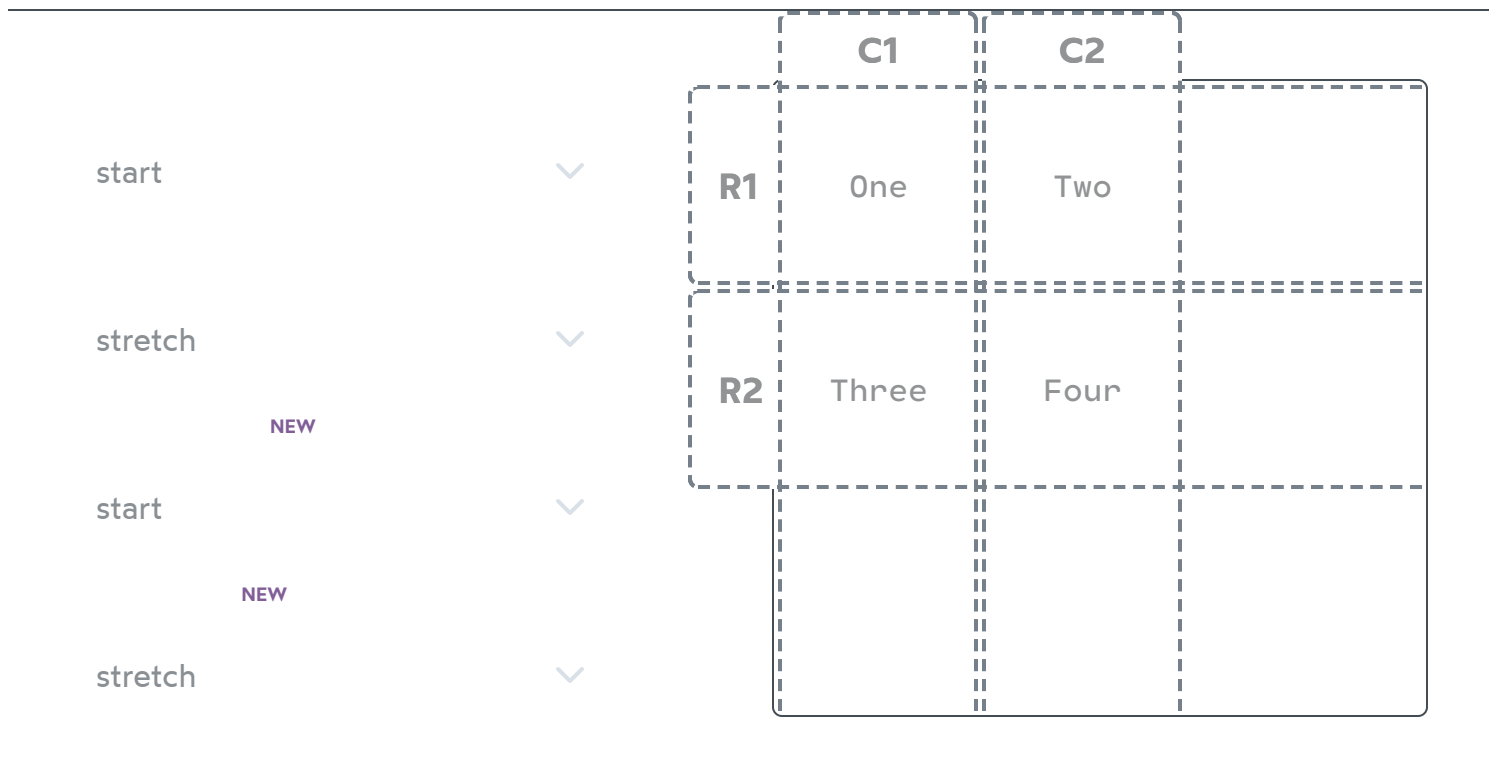


Show Perspective

Unlike `justify-items`, which is set on the grid parent and controls the alignment of *all* grid children, `justify-self` is set on the child. We can think of `justify-items` as a way to set a default value for `justify-self` on all grid children.

Aligning rows

So far, we've been talking about how to align stuff in the *horizontal* direction. CSS Grid provides an additional set of properties to align stuff in the *vertical* direction:



`align-content` is like `justify-content`, but it affects rows instead of columns. Similarly, `align-items` is like `justify-items`, but it handles the *vertical* alignment of items inside their grid area, rather than horizontal.

To break things down even further:

- `justify` — deals with *columns*.
- `align` — deals with *rows*.

- `content` — deals with the *grid structure*.
- `items` — deals with the *DOM nodes* within the grid structure.

Finally, in addition to `justify-self`, we also have `align-self`. This property controls the vertical position of a single grid item within its cell.

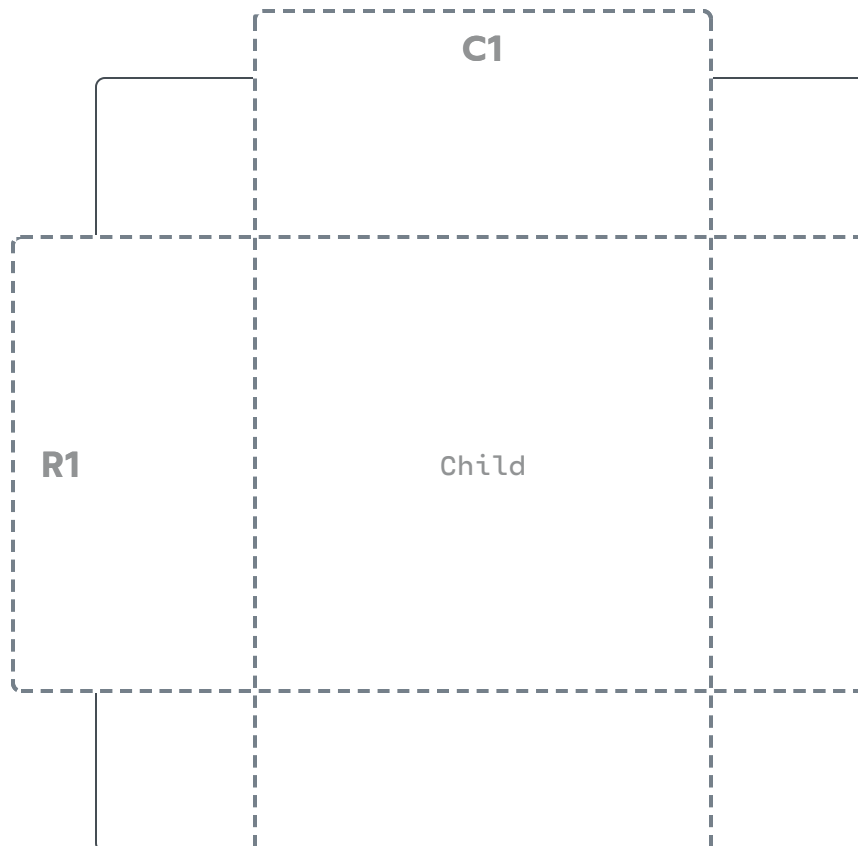
Two-line centering trick

There's one last thing I want to show you. It's one of my favourite little tricks with CSS Grid.

Using only two CSS properties, we can center a child within a container, both horizontally and vertically:

place-content:

start center end



```
.parent {  
  display: grid;  
  place-content: center;  
}
```



Show Perspective

The `place-content` property is a shorthand. It's syntactic sugar for this:

```
.parent {  
  justify-content: center;  
  align-content: center;  
}
```

As we've learned, `justify-content` controls the position of columns. `align-content` controls the position of rows. In this situation, we have an implicit grid with a single child, and so we wind up with a 1×1 grid. `place-content: center` pushes both the row and column to the center.

There are lots of ways to center a div in modern CSS, but this is the only way I know of that only requires two CSS declarations!

Tip of the iceberg

In this tutorial, we've covered some of the most fundamental parts of the CSS Grid layout algorithm, but honestly, there's *so much more stuff* we haven't

talked about!

If you found this blog post helpful, you might be interested to know that I've created a comprehensive learning resource that goes *way deeper*. It's called [_____](#) .

CSS FOR JAVASCRIPT DEVELOPERS

The course uses the same technologies as my blog, and so it's chock full of interactive explanations. But there are also bite-sized videos, practice exercises, real-world-inspired projects, and even a few mini-games.

If you found this blog post helpful, *you'll love the course*. It follows a similar approach, but for the entire CSS language, and with hands-on practice to make sure you're actually developing new skills.

It's specifically built for folks who use a JS framework like React/Angular/Vue. 80% of the course focuses on CSS fundamentals, but we also see how to integrate those fundamentals into a modern JS application, how to structure our CSS, stuff like that.

If you struggle with CSS, I hope you'll check it out. Gaining confidence with CSS is *game-changing*, especially if you're already comfortable with HTML and JS. When you complete the holy trinity, it becomes so much easier to stay in flow, to truly enjoy developing web applications.

You can learn more here:



I hope you found this tutorial useful. ❤️

Last updated on

of hits



Want to know when I publish new content?
Enter your email to join my free newsletter:

marie@yahoo.co.uk



About Josh

CSS for JS Developers

About This Blog

The Joy of React

Contact

Whimsical Animations



© 2018-present Joshua Comeau. All Rights Reserved.

[Terms of Use](#)

[Privacy Policy](#)

[Code of Conduct](#)